

DynamicVLA: A Vision-Language-Action Model for Dynamic Object Manipulation

Haozhe Xie* Beichen Wen* Jiarui Zheng Zhaoxi Chen Fangzhong Hong Haiwen Diao Ziwei Liu ✉

S-Lab, Nanyang Technological University

<https://haozhxie.com/project/dynamic-vla>

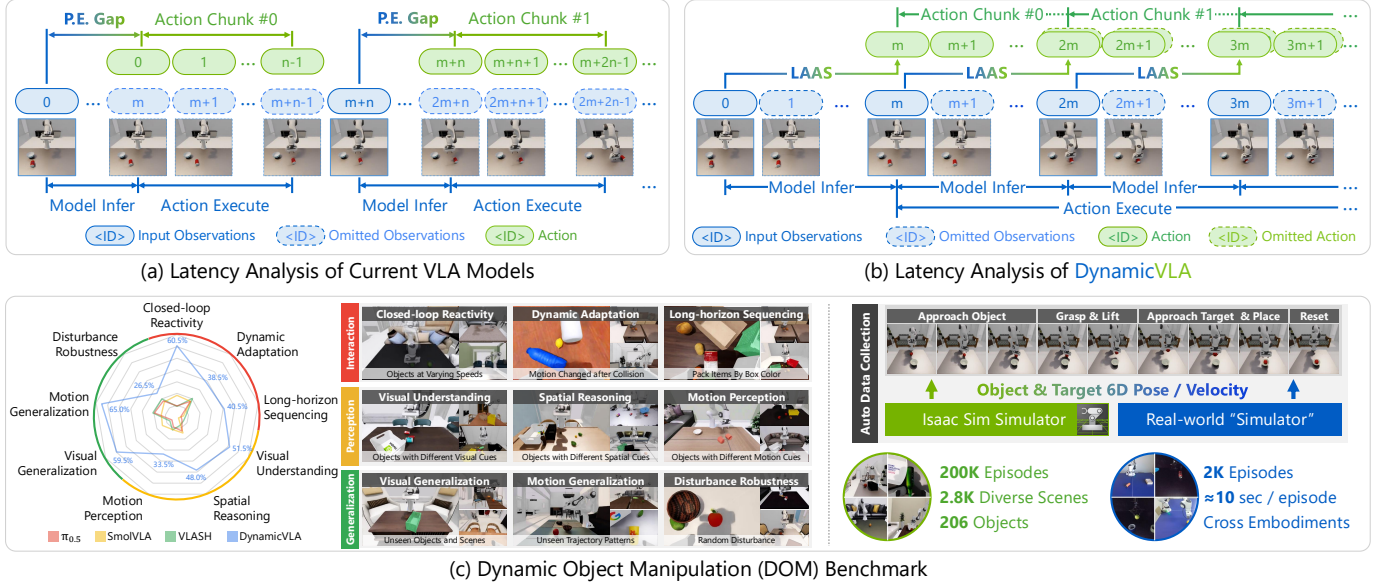


Fig. 1: (a) Current VLA models face perception–execution (P.E.) gaps and inter-chunk waiting, causing delayed reactions to dynamic objects. (b) DynamicVLA addresses these issues through Latent-Aware Action Streaming (LAAS) and Continuous Inference, eliminating both gaps and waiting for seamless action transitions. (c) The Dynamic Object Manipulation (DOM) benchmark, built from scratch, features 2.8K scenes and 206 objects for evaluating Perception, Interaction, and Generalization, while its auto data collection pipeline enables efficient gathering of 200K synthetic and 2K real-world episodes.

Abstract—Manipulating dynamic objects remains an open challenge for Vision-Language-Action (VLA) models, which, despite strong generalization in static manipulation, struggle in dynamic scenarios requiring rapid perception, temporal anticipation, and continuous control. We present **DynamicVLA**, a framework for dynamic object manipulation that integrates temporal reasoning and closed-loop adaptation through three key designs: 1) a compact 0.4B VLA using a convolutional vision encoder for spatially efficient, structurally faithful encoding, enabling fast multimodal inference; 2) Continuous Inference, enabling overlapping reasoning and execution for lower latency and timely adaptation to object motion; and 3) Latent-aware Action Streaming, which bridges the perception–execution gap by enforcing temporally aligned action execution. To fill the missing foundation of dynamic manipulation data, we introduce the Dynamic Object Manipulation (DOM) benchmark, built from scratch with an auto data collection pipeline that efficiently gathers 200K synthetic episodes across 2.8K scenes and 206 objects, and enables fast collection of 2K real-world episodes without teleoperation. Extensive evaluations demonstrate remarkable improvements in response speed, perception, and generalization, positioning DynamicVLA as a unified framework for general dynamic object manipulation across embodiments.

I. INTRODUCTION

Dynamic object manipulation is a fundamental yet underexplored frontier in robotics. Real-world interaction often involves objects in motion, such as handing, repositioning, or stabilizing items, requiring robots to perceive, predict, and act under rapidly changing conditions. Even minor latency may cause task failure, making dynamic manipulation a far more challenging problem than static grasping [4].

To date, robots have been evaluated on moving targets such as throwing, soccer, and table tennis [54, 19, 10], relying on reactive control and handcrafted perception pipelines that operate only in structured settings. Recent works such as DBC-TFP [56] and GEM [22] extend manipulation to moving objects but remain limited to predictable, conveyor-belt-like motion. Meanwhile, concurrent VLA models, including RDT-2 [43], RTVLA [27], and VLASH [39], demonstrate real-time interaction with fast-moving targets, but these tasks tolerant to timing and spatial error. For example, a paddle can return a ball over a wide area, so the interaction does not require the precise 6DoF control needed for dynamic object manipulation.

* Equal Contribution ✉ Corresponding Author

However, open-ended dynamic manipulation, which involves uncertain motion, precise contact, and tight perception–action alignment, remains largely unsolved.

While VLA models [59] have shown strong performance on static manipulation, where object states remain fixed during inference, latency plays only a minor role in such settings. Early VLAs [17, 6] relied on 3B–7B vision-language backbones and still achieved high success rates despite slow inference. More recent designs [38, 46] improve efficiency by reducing model size and increasing throughput while maintaining comparable performance. However, as illustrated in Figure 1a, dynamic manipulation imposes far stricter demands because inference delays desynchronize perception from action and require models to anticipate future object motion, a capability not addressed by prior VLAs for manipulation.

To address these issues, we propose **DynamicVLA**, a framework for dynamic object manipulation that integrates temporal reasoning and closed-loop adaptation through three key designs: **1)** a compact 0.4B-parameter VLA that adopts a convolutional vision encoder for efficient spatial compression and stronger structural preservation, enabling significantly faster and more compact inference in dynamic manipulation settings; **2)** Continuous Inference, a pipelined execution scheme that overlaps prediction and action execution to eliminate inter-chunk waiting and maintain a continuous action stream under dynamic object motion; and **3)** Latent-aware Action Streaming, a latency-aware execution mechanism that restores temporal alignment by discarding outdated actions and prioritizing the most recent predictions at each timestep, ensuring temporally consistent control despite inference delay.

Since existing robotic datasets overwhelmingly capture static scenes and offer no large-scale foundation for dynamic object manipulation, we construct the Dynamic Object Manipulation (DOM) benchmark with a fully automated data-collection pipeline validated across multiple robot embodiments, including Franka Emika Panda and AgileX PiPER. In simulation, Isaac Sim [31] and our task-driven state machine controller use real-time 6D object pose and velocity to drive the robot to manipulate moving objects, producing 200K episodes across 2.8K diverse simulation-ready 3D scenes and 206 objects. Teleoperation is fundamentally ineffective for real-world dynamic manipulation, since fast-moving objects routinely exceed human reaction limits. To address this, we build a real-world “simulator” pipeline that performs 3D object tracking using dual RGB views to estimate 6D pose and infer velocity, and then drives the same state-machine controller to execute autonomous trials, with humans only initiating object motion when necessary.

We extensively evaluate DynamicVLA across dynamic manipulation tasks, multiple robot embodiments, and both simulation and real-world settings, using the DOM benchmark together with 16 real-robot tasks. Our evaluation examines the model’s limits in real-time responsiveness, adaptation to sudden changes in object motion, perceptual grounding of appearance, motion, and spatial descriptions, and generalization to unseen objects, novel scenes, and new motion

regimes. In summary, the contributions of this work consist of a compact 0.4B-parameter VLA tailored for dynamic manipulation, together with two modules that enable real-time closed-loop control. Continuous Inference overlaps inference and execution to eliminate inter-chunk waiting, while Latent-aware Action Streaming enforces temporal alignment between perception and action. We further introduce the DOM benchmark, which supplies large-scale dynamic manipulation data through automated pipelines in both simulation and the real world across multiple robot embodiments.

II. RELATED WORK

Vision-Language-Action Models. Inspired by the success of Large Language Models (LLMs) [36, 40, 1, 41] and Vision-Language Models (VLMs) [25, 26, 42], Vision-Language-Action (VLA) models extend VLMs with action generation. Transformer-based methods [57, 7] use Transformers to model state-action-reward sequences. LLM/VLM-based methods [60, 17] treat VLA tasks as sequence-to-sequence problems for action generation. Diffusion-based methods [9, 32] model policies as denoising diffusion models. LLM and diffusion model methods [14, 6] combine LLMs for representation and diffusion models for action generation. Video generation with inverse kinematics methods [53, 49, 37] generate motion sequences and convert them into actions. However, existing VLA models often suffer from slow inference speeds, limiting their use in scenarios requiring precise or rapid execution.

Robot Learning Datasets. Real-world datasets [28, 45, 35, 50] provide high-fidelity interactions but are costly and hard to scale, while simulated datasets [30, 24, 20, 21, 33] offer scalability yet suffer from the sim-to-real gap. Most benchmarks focus on simple tabletop manipulation (*e.g.*, pick-and-place, pushing) with limited task diversity, though recent work explores long-horizon [24, 55], language-conditioned [58, 16], and tactile-rich [52, 2] settings. Generative models [34, 47, 53, 48] introduce interactive elements but remain constrained by artifacts, low frame rates, and memory. Despite progress in standardization and multi-embodiment learning, current datasets lack dynamic objects, limiting applicability to environments with independent motion.

Robot Dynamic Manipulation. Robotic manipulation has been studied largely in static settings, and existing methods for moving objects remain task-specific or rely on predictable motion. Approaches such as DBC-TFP [56] and GEM [22] operate mainly in structured, conveyor-like scenarios. Concurrent VLA methods, including RDT-2 [43], RTVLA [27], and VLASH [39], demonstrate real-time interaction with fast-moving targets, but these interactions permit large contact margins and do not involve precise 6DoF manipulation. Consequently, general dynamic manipulation under uncertain motion and fine contact constraints remains underexplored.

III. THE DYNAMICVLA MODEL

A. Problem Formulation

We study dynamic object manipulation, where a robot must manipulate objects whose states evolve continuously

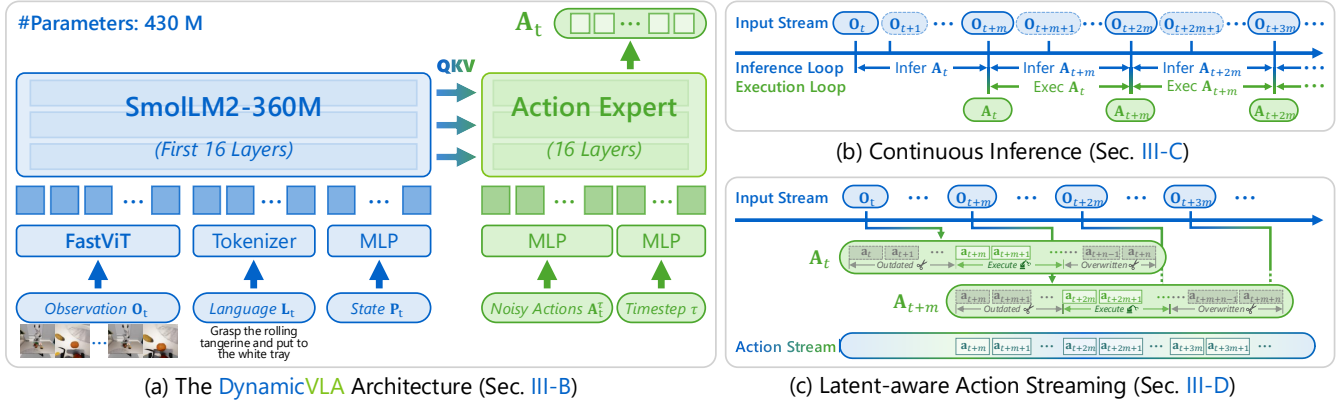


Fig. 2: **Overview of DynamicVLA.** (a) A 0.4B-parameter VLA architecture couples a lightweight backbone with an action expert for fast closed-loop control. (b) Continuous Inference overlaps inference and execution through pipelined inference windows, enabling non-blocking action execution across consecutive action chunks. (c) Latent-aware Action Streaming enforces temporally consistent execution by invalidating outdated actions and prioritizing actions from the most recent action chunk.

during perception, reasoning, and execution. At time step t , the VLA model $\mathcal{M}$ receives a temporal window of visual observations $\mathbf{O}_t = \{\mathbf{o}_{t-k}, \dots, \mathbf{o}_t\}$, a language instruction $\mathbf{L}_t$, and its proprioceptive state $\mathbf{P}_t$, and predicts an action sequence $\mathbf{A}_t = \{\mathbf{a}_t, \dots, \mathbf{a}_{t+n}\}$, i.e., $\mathbf{A}_t = \mathcal{M}(\mathbf{O}_t, \mathbf{L}_t, \mathbf{P}_t)$.

The physical environment includes a latent object state $\mathbf{s}_t$, describing the object’s 6D pose and motion. Crucially, object motion does not pause during inference: while the model reasons over $\mathbf{O}_t$, the object transitions from $\mathbf{s}_t$ to $\mathbf{s}_{t+m}$, where m denotes inference latency, leading to potential misalignment between perception and execution.

B. The DynamicVLA Architecture

Since inference latency directly limits the range of object motion in dynamic manipulation, we design a compact 0.4B VLA model for fast and spatially efficient multimodal reasoning, illustrated in Figure 2a.

Vision–Language Backbone. We adopt SmoLLM2-360M [3] as the language backbone, resulting in an overall tiny model size. Unlike existing VLMs that rely on transformer-based vision encoders, we employ a convolutional vision encoder, FastViT [44], which performs efficient spatial compression and avoids quadratic token growth when processing multi-frame visual inputs. Following SmoVLA [38], we truncate the language backbone to its first 16 transformer layers, significantly reducing inference latency with minimal impact on multimodal reasoning.

Diffusion-Based Action Expert. The action expert $\mathcal{E}_\theta$ predicts an action chunk $\mathbf{A}_t$ conditioned on the multimodal features produced by the VLM backbone. Following diffusion-style action modeling [23, 12], we instantiate $\mathcal{E}_\theta$ as a conditional Flow Matching Transformer [6] and train it using the objective

$$\ell^\tau(\theta) = \mathbb{E}_{p(\mathbf{A}_t|\mathbf{f}_t), q(\mathbf{A}_t^\tau|\mathbf{A}_t)} [\|\mathcal{E}_\theta(\mathbf{A}_t^\tau, \mathbf{O}_t) - \mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t)\|] \quad (1)$$

where superscript $\tau \in [0, 1]$ denotes flow matching timesteps. $q(\mathbf{A}_t^\tau|\mathbf{A}_t) = \mathcal{N}(\tau\mathbf{A}_t, (1-\tau)\mathbf{I})$. $\mathbf{f}_t$ represents the VLM features extracted from $\mathbf{O}_t$, and $\mathbf{A}_t^\tau = \tau\mathbf{A}_t + (1-\tau)\epsilon$, with

$\epsilon \sim \mathcal{N}(0, \mathbf{I})$. Under this objective, $\mathcal{E}_\theta(\mathbf{A}_t^\tau, \mathbf{O}_t)$ learns to match the denoising vector field $\mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t) = \epsilon - \mathbf{A}_t$.

Multi-modal Fusion and Projection. We employ lightweight linear projections to align representations across modules, including 1) embedding robot states into the multimodal feature space, 2) adapting action representations to the diffusion-based action expert, and 3) matching the output dimensions between the VLM backbone and the action expert.

C. Continuous Inference

At time step t , the VLA model $\mathcal{M}$ predicts an action sequence $\mathbf{A}_t = \{\mathbf{a}_t, \dots, \mathbf{a}_{t+n}\}$. In existing VLA models [18, 6, 15], a new inference is triggered only after the previously predicted action sequence $\mathbf{A}_t$ is fully executed. This serializes inference and execution, introducing inter-chunk waiting that stalls control until the next action sequence is available and degrades responsiveness under dynamic object motion.

Under Continuous Inference, inference cycles are triggered as soon as the previous inference finishes, independent of whether the previously predicted action sequence has been exhausted, as shown in Figure 2b. Let m denote the inference delay, i.e., the number of timesteps between the start and completion of an inference cycle. Inference therefore completes at timesteps $t, t+m, t+2m, \dots$ where m may vary across cycles; for clarity, we assume a constant m in the formulation.

During execution, actions from $\mathbf{A}_t$ are executed continuously while the next action sequence $\mathbf{A}_{t+m}$ is being inferred. We assume $n > m$, such that a new action sequence becomes available before the execution of the current sequence completes. Consequently, execution does not block on inference completion, eliminating inter-chunk waiting.

D. Latent-aware Action Streaming

As presented in Figure 2c, inference delay m introduces temporal misalignment between predicted actions and the evolving environment, which manifests in two ways:

1) Perception–Execute Gap: when inference is initiated at time t to predict $\mathbf{A}_t$, the predicted actions become available

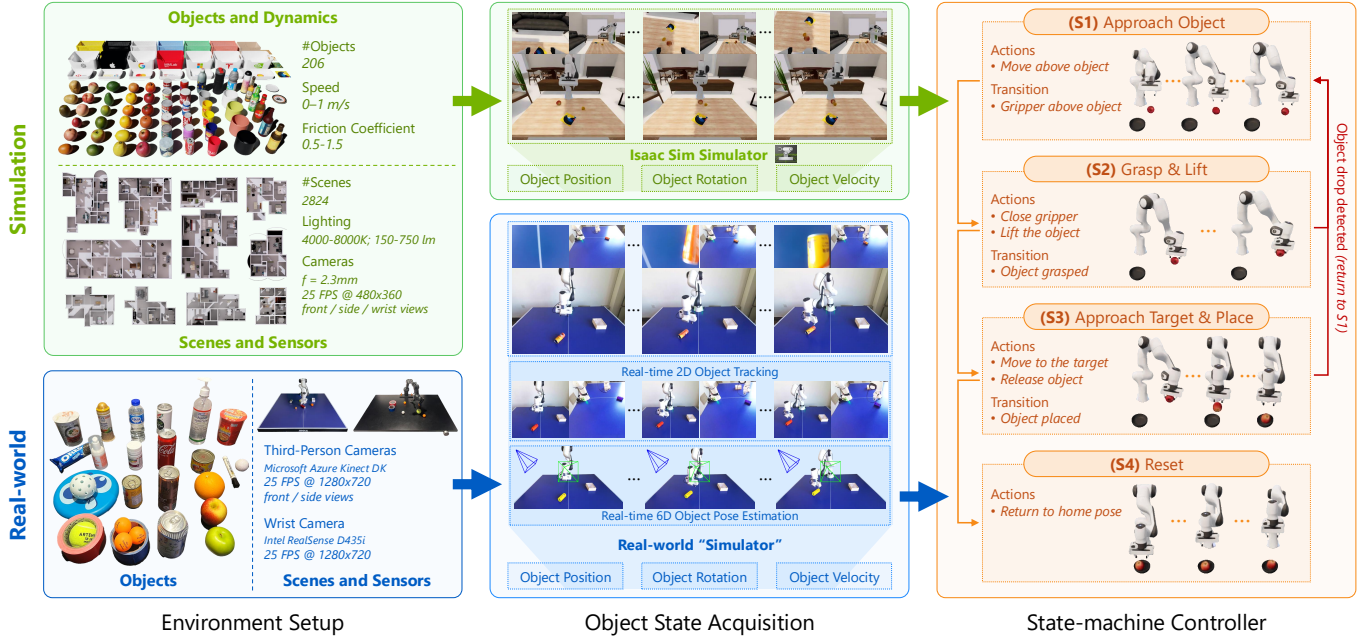


Fig. 3: **Automatic Simulation and Real-world Data Collection.** *Environment Setup*: simulation and real-world settings share diverse objects, tabletop scenes, and synchronized multiview cameras. *Object State Acquisition*: simulation provides ground-truth 6D object states, while real-world multiview RGB observations are converted into a real-world “simulator” interface that enables automatic dynamic-manipulation data collection without teleoperation or ground-truth sensing. *State-machine Controller*: a shared four-stage controller uses these states to execute approach, grasp, place, and reset behaviors.

only at $t + m$, by which time the observation has evolved to $\mathbf{O}_{t+m}$. Consequently, actions $\{\mathbf{a}_t, \dots, \mathbf{a}_{t+m-1}\}$ are no longer aligned with the current observation.

2) Conflicts Between Overlapping Action Chunks: continuous inference allows a new action sequence $\mathbf{A}_{t+m}$ to be generated before the execution of $\mathbf{A}_t$ is complete, resulting in multiple candidate actions for the same execution timestep.

Latent-aware Action Streaming resolves both issues through an explicit execution strategy. Specifically, actions in $\mathbf{A}_t$ corresponding to timesteps earlier than $t + m$ are discarded as outdated, and execution proceeds with the subsequence $\{\mathbf{a}_{t+m}, \dots, \mathbf{a}_{t+n}\}$. For timesteps where $\mathbf{A}_t$ and $\mathbf{A}_{t+m}$ overlap, actions from the newer sequence $\mathbf{A}_{t+m}$ are prioritized, overwriting those from $\mathbf{A}_t$, allowing execution to adapt promptly to the most recent environment state, particularly under dynamic object motion.

IV. THE DYNAMIC OBJECT MANIPULATION BENCHMARK

A. Overview

Dynamic Object Manipulation (DOM) is the first large-scale benchmark dedicated to dynamic object manipulation, addressing the lack of standardized datasets for evaluating robotic policies on moving objects. DOM provides scalable data collection through fully automated pipelines in both simulation and the real world, producing 200K synthetic episodes and 2K real-world episodes, where teleoperation is ineffective due to human reaction limits under fast object motion. The benchmark organizes dynamic manipulation scenarios along

structured interaction, perception, and generalization dimensions, enabling consistent and comparable evaluation across algorithms and robot embodiments.

B. Benchmark Dimensions

As illustrated in Figure 1c, DOM evaluates dynamic manipulation ability across three principal dimensions:

Interaction. This dimension evaluates how effectively a policy responds to evolving object motion. 1) *Closed-loop reactivity*, which measures how quickly the robot adjusts to objects moving at different speeds; 2) *Dynamic adaptation*, where the policy must handle abrupt changes in motion such as direction shifts or unexpected disturbances; 3) *Long-horizon sequencing*, which assesses whether the policy maintains coherent behavior over extended interactions and prioritizes actions as motion events unfold.

Perception. This dimension evaluates how well a policy perceives and grounds visual and linguistic cues in dynamic environments. 1) *Visual understanding*, which measures the ability to distinguish objects with similar shapes, textures, or materials; 2) *Spatial reasoning*, which examines whether the policy can infer object positions and relative arrangements in cluttered or changing scenes; 3) *Motion perception*, which assesses how accurately the policy interprets object motion cues such as speed and direction.

Generalization. This dimension evaluates how robustly a policy transfers across novel objects, scenes, and motion patterns. 1) *Visual generalization*, which measures adaptation to unseen shapes, appearances, and scene layouts; 2) *Motion*

generalization, which assesses whether the policy can handle new speed ranges, altered friction conditions, and trajectory patterns that differ from those observed during training; 3) *Disturbance Robustness*, which tests the ability to maintain stable behavior under external perturbations such as unexpected pushes, collisions, or sensor noise.

C. Simulation Data Collection

Our simulation framework is designed with two core objectives: 1) to rapidly scale up dynamic manipulation data for pretraining VLA policies, and 2) to produce a reproducible and standardized benchmark that supports fair and consistent evaluation across future work. As shown in Figure 3, we construct a high-throughput pipeline in Isaac Sim [31] that unifies scene and object sampling, multi-view perception, real-time object-state acquisition, and closed-loop control.

Objects and Dynamics. We include 206 everyday objects from Objaverse [11] spanning fruits, vegetables, containers, and other household items, with texture augmentation for additional visual diversity. Object speeds are sampled from 0–0.75 m/s (with some remaining static) and friction coefficients from 0.5–1.5. Multiple objects are placed in the workspace, allowing natural interactions during motion.

Scenes and Sensors. We derive 2.8K diverse 3D scenes from 3D-FRONT [13], curated to ensure a clean, flat tabletop and to remove self-occluding or unrealistic object placements. Each scene is instrumented with three cameras: two third-person views placed 1 m from the robot (front at 0.6 m height and left at 0.35 m height) and a wrist-mounted camera. All cameras capture RGB frames at 25 FPS with a 480×360 resolution, using a 2.3 mm focal length aligned with Azure Kinect intrinsics. We randomize scene illumination by sampling color temperature from 4000–8000 K, light intensity from 150–750 lm, and light source positions from $x \in [-50, 50]$ m, $y \in [-50, 50]$ m, $z \in [10, 20]$ m.

Object State Acquisition. The simulator maintains ground-truth 6D object states throughout each episode. Isaac Sim randomizes physical parameters and propagates object motion via the physics engine, from which we extract per-object position, rotation, and linear/angular velocity at 25 Hz. The resulting noise-free trajectories provide the controller with real-time motion cues for short-horizon prediction and state transitions. This interface is later mirrored in the real-world pipeline to ensure consistent behavior across embodiments.

State Machine Controller. The state machine consumes real-time 6D object pose, velocity, and the 6D pose of a static target object, and executes a four-stage closed-loop routine: 1) *Approach Object*: predict near-future object motion (about 0.23 s) and position the end effector 10 cm above the predicted location with continuous updates. 2) *Grasp & Lift*: descend, stabilize residual motion, and secure a grasp before lifting. 3) *Approach Target & Place*: move toward the placement pose derived from the target object’s 6D geometry and place the object accurately. 4) *Reset*: return to the home pose to begin the next episode. This design produces reactive, prediction-

informed trajectories, enabling scalable generation of realistic dynamic manipulation episodes.

D. Real-World Data Collection

Teleoperation is widely used for collecting demonstrations, but it breaks down for dynamic manipulation: human reaction is too slow to track fast-moving objects, even with homomorphic interfaces. Meanwhile, the real world lacks ground-truth 6D object states, making the simulator’s closed-loop pipeline impossible to replicate directly. To address both issues, we build a real-world “simulator”—a high-frequency perception and state-estimation system that approximates simulator-style object states using commodity RGB-D sensors and enables fast (≈ 10 s per episode), teleoperation-free collection of large-scale dynamic manipulation data that runs identically on Franka and PiPER for consistent multi-embodiment coverage.

Environment Setup. We use 25 physical household objects spanning containers, food items, bottles, and tools, with multiple objects per episode, including pick/place targets and natural distractors. The scene is captured by two synchronized third-person RGB cameras (Azure Kinect DK) placed at front and side viewpoints, along with a wrist-mounted RealSense D435i, matching the simulation geometry and supplying synchronized, calibrated RGB streams for state estimation.

Object State Acquisition. To replicate the simulator’s state interface, we build a “real-time” simulator that outputs 6D object pose and velocity. EfficientTAM [51] supplies per-view object masks from the synchronized third-person cameras, and a geometric triangulation step recovers the 3D centroid. Linear and angular velocities are obtained by fitting motion over a short temporal window, producing a smooth, low-latency 6D state stream compatible with the controller’s requirements.

State-machine Controller. The same four-stage controller used in simulation runs unchanged in the real world, consuming the estimated 6D object states and target pose.

V. EXPERIMENTS

Our experiments evaluate DynamicVLA on dynamic object manipulation under real-time constraints. We benchmark DynamicVLA against representative VLA baselines across a wide range of dynamic manipulation scenarios, covering interaction, perception, and generalization challenges. Moreover, we analyze the impact of key system components and the trade-offs between model capacity and inference efficiency. Specifically, we study the following research questions:

- 1) How well can DynamicVLA interact with fast-moving objects and maintain stable closed-loop behavior over long horizons?
- 2) How reliably does DynamicVLA interpret appearance, spatial, and motion cues during dynamic manipulation?
- 3) How well does DynamicVLA generalize to unseen objects, novel 3D scenes, and unseen motion regimes?
- 4) How do the key components affect performance, and what trade-offs arise between model capacity and inference efficiency?

TABLE I: **Dynamic Object Manipulation Simulation Benchmark Results.** Average success rates (SR, %) are reported across nine evaluation sub-dimensions, organized under three categories: Interaction, Perception, and Generalization. In addition, overall average SR (%), path length (Path Len, meters), and task completion time (Time, seconds) are reported. Each method is evaluated over 1,800 trials (10 scenes $\times$ 9 dimensions $\times$ 20 trials). All baseline models are fine-tuned on the DOM dataset using their official implementations and released pretrained weights. Best results are highlighted in bold.

Methods	Interaction			Perception			Generalization			Average		
	CR	DA	LS	VU	SR	MP	VG	MG	DR	SR $\uparrow$	Path Len $\downarrow$	Time $\downarrow$
Diffusion Policy [9]	0.50	0.50	0.00	1.00	0.00	0.00	1.00	0.50	0.00	0.38	1.34	10.89
OpenVLA-OFT [18]	3.50	0.50	0.50	0.00	1.50	0.50	3.50	2.00	0.00	1.33	1.08	10.83
π_0 [6]	7.50	12.00	3.00	5.50	10.50	7.50	5.50	12.50	9.00	8.11	1.19	10.55
$\pi_{0.5}$ [15]	9.50	17.50	3.50	5.00	12.50	9.00	5.00	19.50	18.00	11.06	1.28	10.62
SmolVLA [38]	18.50	17.50	5.50	1.50	14.50	11.50	14.50	13.50	17.00	12.67	1.30	10.65
GR00T-N1.5 [5]	10.50	12.00	4.00	9.50	13.50	14.00	14.50	19.50	20.00	13.05	1.29	10.56
VLA-Adapter-Pro [46]	21.00	15.50	6.00	6.50	16.50	10.50	15.00	18.50	13.00	13.61	1.51	9.98
VLASH [39]	9.00	20.50	7.50	6.50	7.50	12.00	7.00	21.00	20.00	12.33	1.27	10.60
DynamicVLA	60.50	38.50	40.50	51.50	48.00	33.50	59.50	65.00	26.50	47.06	2.50	8.53

CR: Closed-loop Reactivity; DA: Dynamic Adaptation; LS: Long-horizon Sequencing; VU: Visual Understanding; SR: Spatial Reasoning; MP: Motion Perception; VG: Visual Generation; MG: Motion Generalization; DR: Disturbance Robustness

A. Evaluation Protocols

Experimental Setup. DynamicVLA is evaluated in both simulation and real-world settings on the Dynamic Object Manipulation (DOM) benchmark (Sec. IV). Experiments are conducted in three environments: Isaac Sim with a Franka Emika Panda arm, a real-world Franka arm, and a real-world AgileX PIPER arm, covering both simulated and physical embodiments. For fair comparison in dynamic settings, object motion is standardized across methods using a secondary robot arm following a fixed launching trajectory. Although initial velocities vary due to physical noise, motion patterns remain comparable across trials. Each real-world experiment is repeated 20 times, and results are averaged. All methods are evaluated under identical conditions within each environment.

Baselines. In simulation, we evaluate Diffusion Policy [9], OpenVLA-OFT [18], π_0 [6], $\pi_{0.5}$ [15], SmolVLA [38], GR00T-N1.5 [5], VLA-Adapter-Pro [46], and VLASH [39], covering general-purpose VLAs, lightweight adaptation-based models, and latency-aware designs. In real-world experiments, we evaluate $\pi_{0.5}$, SmolVLA, and VLASH under identical physical setups. All baselines are initialized from publicly available pretrained weights and adapted to the DOM benchmark using a consistent fine-tuning protocol.

Evaluation Metrics. All methods are evaluated using three metrics: 1) *Success Rate*, the fraction of trials that complete the instructed manipulation without object drop or timeout; 2) *Path Length*, the total end-effector trajectory length during execution; 3) *Task Completion Time*, the elapsed time from the onset of object motion to task termination, including successful completion, timeout, or object drop. In simulation, we report all three metrics. In real-world experiments, we report the success rate. All metrics are averaged over multiple trials.

Execution Constraints. To ensure safe real-world operation, we restrict the robot workspace within predefined bounds. If the predicted end-effector position exceeds a predefined safety threshold, the robot aborts the current attempt and returns to a safe home pose, and the trial is marked as failure.

B. Dynamic Interaction and Reactivity

We analyze the *Interaction* dimension of the DOM benchmark, which evaluates closed-loop reactivity, dynamic adaptation, and long-horizon sequencing in dynamic object manipulation. These settings progressively increase in difficulty, from reacting to speed-varying motion, to recovering from abrupt event-driven changes, and finally sustaining coordination over extended interactions with multiple moving objects.

Across all three interaction settings (Table I, Interaction–CR/DA/LS), prior VLAs exhibit consistently low success under dynamic motion, while DynamicVLA maintains robust performance. Specifically, DynamicVLA achieves 60.5/38.5/40.5% success, outperforming the strongest baseline by +188.1/+87.8/+440.0% across all interaction settings. This trend is consistent in real-world experiments (Figure 4), where baseline methods frequently fail due to delayed reactions, stale action execution, or loss of coordination, whereas DynamicVLA more reliably re-aligns perception and action under tight temporal constraints.

C. Multimodal Spatial-Temporal Reasoning

We evaluate the *Perception* dimension of the DOM benchmark, which probes vision-language reasoning under dynamic manipulation. This dimension increases progressively in difficulty, moving from visual recognition to spatial reasoning and finally motion perception, each placing greater demands on the underlying VLM.

As shown in Table I (Perception–VU/SR/MP), performance degrades consistently as the task shifts along this progression. Although many VLAs perform well in static manipulation, their performance drops markedly under dynamic scenes, with particularly sharp degradation in spatial and motion reasoning as evolving spatial-temporal relationships require timely and accurate interpretation. This limitation is further exacerbated by strict real-time and model-size constraints: to meet interaction latency requirements, lightweight VLAs must compromise VLM capacity, making perception-heavy dynamic tasks

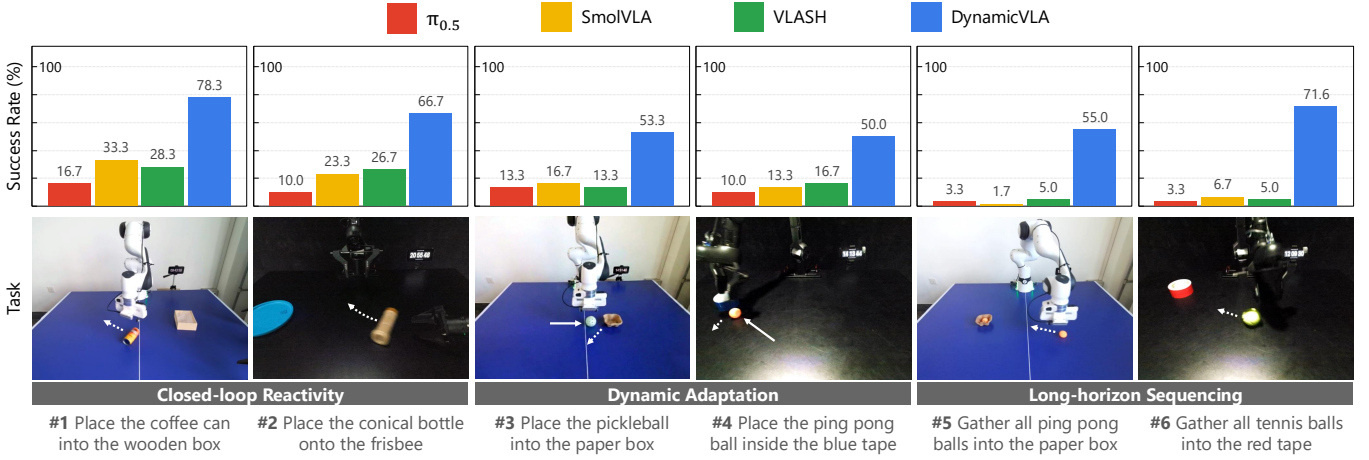


Fig. 4: **Real-world Interaction Evaluation.** We compare representative VLA models on six real-world dynamic manipulation tasks across Franka and PiPER, averaging success rates over 20 trials for each of three paired motion–position configurations, with object motion generated by a secondary robot arm.

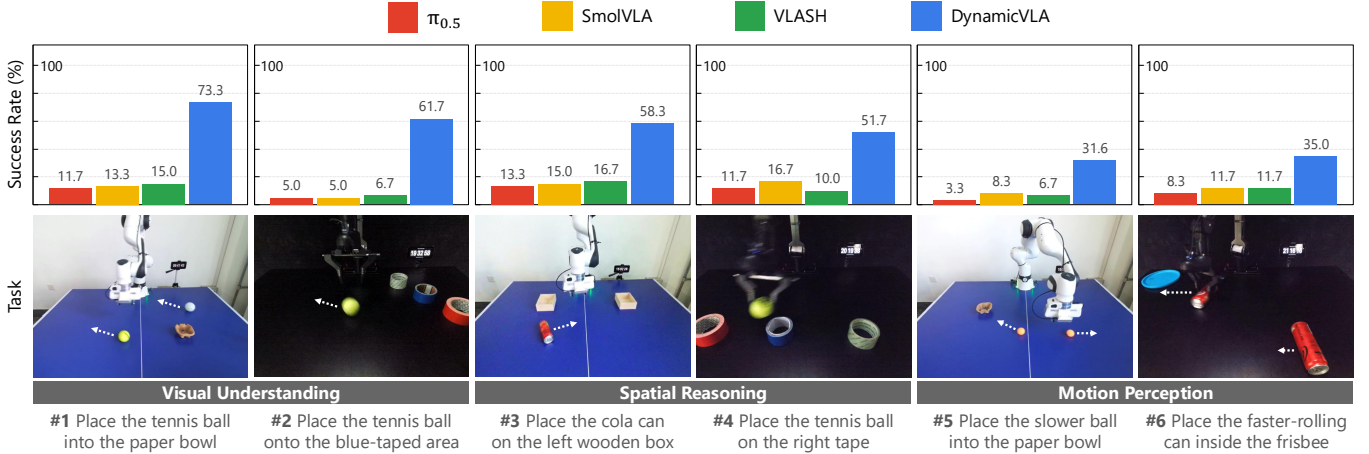


Fig. 5: **Real-world Perception Evaluation.** We compare representative VLA models on six real-world dynamic manipulation tasks across Franka and PiPER, averaging success rates over 20 trials for each of three paired motion–position configurations, with object motion generated by a secondary robot arm.

especially challenging. This trend is consistently reflected in real-world experiments (Figure 5), where the best baselines, achieve lower success (11.7%) due to frequent spatial-temporal misalignment, whereas DynamicVLA reaches 51.9% success.

D. Generalization to Unseen Frontiers

We examine the *Generalization* dimension of the DOM benchmark, which assesses a policy’s robustness to distribution shifts beyond training conditions. This dimension comprises three complementary aspects, targeting appearance variation, unseen motion patterns, and environmental perturbations.

As shown in Table I (Generalization–VG/MG/DR), prior VLAs exhibit low success under distribution shifts in appearance, motion, and environmental perturbations, while DynamicVLA achieves higher overall performance. Similar trends are observed in real-world experiments (Figure 6) for appearance and motion shifts. In contrast, robustness to environmental perturbations remains challenging even for DynamicVLA.

This setting involves stronger perturbations in simulation that go beyond idealized physical assumptions. We therefore omit real-world results, as such perturbations are difficult to reproduce reliably and their prevalence in physical environments (*e.g.*, surface irregularities) is hard to control.

E. Ablation Studies

To evaluate the impact of design choices in DynamicVLA, we perform ablation studies isolating model capacity, visual encoding, and execution mechanisms. All variants are evaluated on the DOM benchmark under identical training protocols and metrics, with results summarized in Table II.

Backbone Capacity. To evaluate the effect of language model capacity, we compare SmolLM2 [3] backbones of different sizes (135M, 360M, and 1.7B) under the same architecture and execution setup. Increasing model size improves representational capacity but also incurs higher inference latency, which degrades closed-loop responsiveness and leads to lower suc-

TABLE II: **Ablation of key design choices.** The effects of LLM backbone size (Size), the use of FastViT as the vision encoder (FViT), Continuous Inference (CI), and Latent-aware Action Streaming (LAAS) are evaluated by reporting success rate (SR), path length (PL), and task completion time (Time) on the DOM benchmark. The final row corresponds to the DynamicVLA model configuration.

	Size	FViT	CI	LAAS	SR (%) $\uparrow$	PL (m) $\downarrow$	Time (s) $\downarrow$
[1]	360M	✓	✗	✗	30.27	2.77	9.86
[2]	360M	✓	✗	✓	36.11	1.77	9.51
[3]	360M	✓	✓	✗	39.72	2.61	8.84
[4]	135M	✓	✓	✓	26.67	1.82	9.95
[5]	1.7B	✓	✓	✓	24.33	1.77	9.91
[6]	360M	✗	✓	✓	28.89	1.86	9.89
[7]	360M	✓	✓	✓	47.06	2.50	8.53

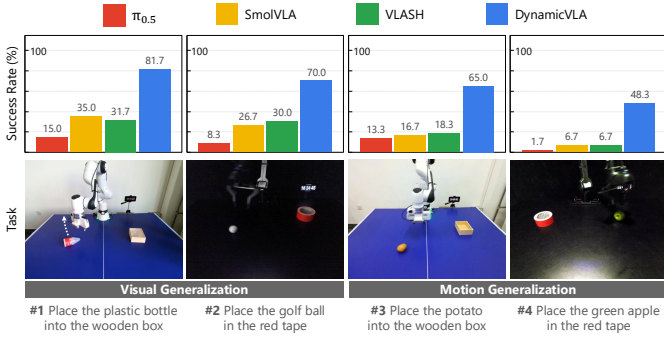


Fig. 6: **Real-world Generation Evaluation.** We compare representative VLA models on four real-world dynamic manipulation tasks across Franka and PiPER, averaging success rates over 20 trials for each of three paired motion–position configurations, with object motion generated by a secondary robot arm.

cess rates in dynamic scenarios. Conversely, reducing model size improves inference speed but limits reasoning capacity, resulting in suboptimal action prediction. As shown in Table II ([4], [5], and [7]), the 360M model achieves the best balance between inference efficiency and model capacity, yielding the highest overall performance in dynamic object manipulation.

Vision Encoder. We ablate the choice of visual encoder by replacing the convolutional FastViT encoder with a transformer-based vision encoder, implemented using the same configuration as in SmolVLM [29], while keeping all other components fixed. As indicated in Table II ([6] and [7]), FastViT outperforms transformer-based encoders by lowering encoding latency through reduced tokenization, while maintaining structurally faithful visual representations.

Continuous Inference. To demonstrate the effectiveness of Continuous Inference (CI), we disable it while keeping all other components unchanged (Table II, [2] and [7]). Without CI, inference is triggered only after the previous action chunk is fully executed, introducing inter-chunk waiting that degrades responsiveness and leads to lower success rates and longer completion times in dynamic manipulation tasks.

Latent-aware Action Streaming. We further analyze the

contribution of Latent-aware Action Streaming (LAAS) under Continuous Inference by disabling it while retaining CI (Table II, [3] and [7]). Despite enabling continuous action generation, CI alone remains insufficient under inference delay, as temporal misalignment between predicted actions and the evolving environment degrades performance. LAAS resolves this issue by discarding outdated actions and prioritizing the most recent predictions, enforcing temporally aligned execution and improving stability in dynamic scenarios. Comparing [1] and [7] reveals a more severe performance drop when both CI and LAAS are disabled, indicating that they play complementary roles in dynamic manipulation.

VI. DISCUSSION AND FUTURE WORK

This work shows that, for dynamic object manipulation with VLA models, the dominant failure mode is not perceptual ambiguity but the temporal misalignment between observation and action execution — a factor largely ignored in static manipulation. To address this misalignment, we design **DynamicVLA** with three innovations: 1) a compact 0.4B backbone that supports high-frequency reasoning; 2) Continuous Inference to overlap reasoning and execution for timely adaptation; 3) Latent-aware Action Streaming to enforce temporally aligned action execution. To address the scarcity of large-scale dynamic manipulation data, we develop an automatic simulation and real-world data collection pipeline that drives a robot arm with state-machine controllers, using object states from the simulation engine and from a real-world “simulator” interface, respectively. Together, these elements significantly reduce the perception–execution gap and yield more responsive behavior than conventional VLA models.

Looking forward, several limitations of the current study point to promising directions for future work:

More Efficient VLA Architectures. While DynamicVLA highlights the importance of latency-aware design for dynamic manipulation, real-time constraints fundamentally trade off multimodal understanding against responsiveness. Dynamic tasks tightly couple perception, reasoning, and execution, demanding architectures and inference schemes that preserve understanding under strict latency budgets.

Beyond Short-horizon Dynamics. Our current formulation emphasizes short- to medium-horizon reactive interaction, which exposes latency-induced failures but does not capture longer-horizon dynamic behaviors. Future work should extend dynamic manipulation to multi-stage tasks with persistent object motion, integrating planning, memory, and task decomposition while remaining compatible with language conditioning and real-time execution constraints.

Beyond Rigid-Body Dynamics. Our data pipeline assumes rigid-body state estimation, whereas many dynamic tasks involve non-rigid or fluid dynamics with continuously evolving states that are difficult to represent in both simulation and the real world. Extending VLA models and data pipelines to such settings remains an open challenge.

ACKNOWLEDGMENTS

We thank Prof. David Hsu (NUS) and Prof. Shengping Zhang (HIT) for their support and for providing access to the Franka Emika Panda robot utilized in this work. This work was supported by the Singapore Ministry of Education under its Academic Research Fund Tier 2 (MOE-T2EP20221-0012, MOE-T2EP20223-0002), and by cash and in-kind contributions from NTU S-Lab and industry partner(s).

REFERENCES

- [1] DeepSeek AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv*, 2501.12948, 2025.
- [2] Ireaiyo Akinola, Jie Xu, Jan Carus, Dieter Fox, and Yashraj Narang. TacSL: A library for visuotactile sensor simulation and learning. *IEEE T-RO*, 41:2645–2661, 2025.
- [3] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. SmoLM2: when smol goes big - data-centric training of a small language model. *arXiv* 2502.02737, 2025.
- [4] Shuanghao Bai, Wenxuan Song, Jiayi Chen, Yuheng Ji, Zhide Zhong, Jin Yang, Han Zhao, Wanqi Zhou, Wei Zhao, Zhe Li, Pengxiang Ding, Cheng Chi, Haoang Li, Chang Xu, Xiaolong Zheng, Donglin Wang, Shanghang Zhang, and Badong Chen. Towards a unified understanding of robot manipulation: A comprehensive survey. *arXiv* 2510.10903, 2025.
- [5] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi, Yu Fang, Dieter Fox, Fengyuan Hu, and Spencer Huang et al. GR00T N1: an open foundation model for generalist humanoid robots. *arXiv* 2503.14734, 2025.
- [6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, and Brian Ichter et al. π_0 : A Vision-Language-Action flow model for general robot control. In *RSS*, 2025.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, and Jasmine Hsu et al. RT-1: robotics transformer for real-world control at scale. In *RSS*, 2023.
- [8] Minwoo Byeon, Beomhee Park, Haecheon Kim, Sungjun Lee, Woonhyuk Baek, and Saehoon Kim. COYO-700M: image-text pair dataset. <https://github.com/kakaobrain/coyo-dataset>, 2022.
- [9] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.
- [10] David B. D’Ambrosio, Saminda Abeyruwan, Laura Graesser, Atil Iscen, Heni Ben Amor, Alex Bewley, Barney J. Reed, Krista Reymann, Leila Takayama, and Yuval Tassa et al. Achieving human level competitive robot table tennis. *arXiv*, 2408.03906, 2024.
- [11] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3D objects. In *CVPR*, 2023.
- [12] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *ICML*, 2024.
- [13] Huan Fu, Bowen Cai, Lin Gao, Lingxiao Zhang, Jiaming Wang, Cao Li, Qixun Zeng, Chengyue Sun, Rongfei Jia, Binqiang Zhao, and Hao Zhang. 3D-FRONT: 3D furnished rooms with layouts and semantics. In *ICCV*, 2021.
- [14] Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Quan Vuong, Ted Xiao, Pannag R. Sanketi, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *RSS*, 2024.
- [15] Physical Intelligence. $\pi_0.5$: a Vision-Language-Action model with open-world generalization. In *CoRL*, 2025.
- [16] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. VIMA: general robot manipulation with multimodal prompts. *arXiv* 2210.03094, 2022.
- [17] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source Vision-Language-Action model. *arXiv*, 2406.09246, 2024.
- [18] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning Vision-Language-Action models: Optimizing speed and success. In *RSS*, 2025.
- [19] Hiroaki Kitano, Minoru Asada, Yasuo Kuniyoshi, Itsuki Noda, and Eiichi Osawa. RoboCup: the robot world cup initiative. In *International Conference on Autonomous Agents*, 1997.
- [20] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, and Benjamin Jose Martinez et al. BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1, 000 everyday activities

- and realistic simulation. *arXiv*, 2403.09227, 2024.
- [21] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. In *CoRL*, 2024.
 - [22] Zhuoling Li, Xiaoyang Wu, Zhenhua Xu, and Hengshuang Zhao. Train once, deploy anywhere: Realize data-efficient dynamic object manipulation. *arXiv*, 2508.14042, 2025.
 - [23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *ICLR*, 2023.
 - [24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS*, 2023.
 - [25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.
 - [26] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *CVPR*, 2024.
 - [27] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running VLAs at real-time speed. *arXiv* 2510.26742, 2025.
 - [28] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Booher, Max Spero, Albert Tung, Julian Gao, John Emmons, Anchit Gupta, Emre Orbay, Silvio Savarese, and Li Fei-Fei. ROBOTURK: A crowdsourcing platform for robotic skill learning through imitation. In *CoRL*, 2018.
 - [29] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. SmolVLM: redefining small and efficient multimodal models. *arXiv* 2504.05299, 2025.
 - [30] Oier Mees, Lukás Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE RA-L*, 7(3):7327–7334, 2022.
 - [31] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrügg, and Nikita Rudin et al. Isaac Lab: A gpu-accelerated simulation framework for multimodal robot learning. *arXiv* 2511.04831, 2025.
 - [32] Percy Liang Moo Jin Kim, Chelsea Finn. Fine-tuning Vision-Language-Action models: Optimizing speed and success. In *RSS*, 2025.
 - [33] Yao Mu, Tianxing Chen, Zanzin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, Lunkai Lin, Zhiqiang Xie, Mingyu Ding, and Ping Luo. RoboTwin: dual-arm robot benchmark with generative digital twins. In *CVPR*, 2025.
 - [34] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *RSS*, 2024.
 - [35] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, and Ajinkya Jain et al. Open X-Embodiment: robotic learning datasets and RT-X models. In *ICRA*, 2024.
 - [36] OpenAI. GPT-4 technical report. *arXiv*, 2303.08774, 2023.
 - [37] Zhongwei Ren, Yunchao Wei, Xun Guo, Yao Zhao, Bingyi Kang, Jiashi Feng, and Xiaojie Jin. VideoWorld: Exploring knowledge learning from unlabeled videos. In *CVPR*, 2025.
 - [38] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andrés Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Rémi Cadène. SmolVLA: A Vision-Language-Action model for affordable and efficient robotics. *arXiv* 2506.01844, 2025.
 - [39] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: real-time VLAs via future-state-aware asynchronous inference. *arXiv* 2512.01031, 2025.
 - [40] Llama Team. The llama 3 herd of models. *arXiv*, 2407.21783, 2024.
 - [41] Qwen Team. Qwen2.5-1M technical report. *arXiv*, 2501.15383, 2025.
 - [42] Qwen Team. Qwen2.5-VL technical report. *arXiv*, 2502.13923, 2025.
 - [43] RDT Team. RDT2: enabling zero-shot cross-embodiment generalization by scaling up UMI data, September 2025. URL <https://github.com/thu-ml/RDT2>.
 - [44] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. FastViT: A fast hybrid vision transformer using structural reparameterization. In *ICCV*, 2023.
 - [45] Homer Rich Walke, Kevin Black, Tony Z. Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, Abraham Lee, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *CoRL*, 2023.
 - [46] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, Siteng Huang, Yifan Tang, Wenhui Wang, Ru Zhang, Jianyi Liu, and Donglin Wang. VLA-Adapter: an effective paradigm for tiny-scale Vision-Language-Action model. *arXiv* 2509.09372, 2025.
 - [47] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. RoboGen: Towards unleashing infinite data for automated robot learning via

- generative simulation. In *ICML*, 2024.
- [48] Beichen Wen, Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. 3D scene generation: A survey. *arXiv 2505.05474*, 2025.
 - [49] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *ICLR*, 2024.
 - [50] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, Yinuo Zhao, Zhiyuan Xu, and Guang Yang et al. RoboMIND: benchmark on multi-embodiment intelligence normative data for robot manipulation. In *RSS*, 2025.
 - [51] Yunyang Xiong, Chong Zhou, Xiaoyu Xiang, Lemeng Wu, Chenchen Zhu, Zechun Liu, Saksham Suri, Balakrishnan Varadarajan, Ramya Akula, Forrest N. Iandola, Raghuraman Krishnamoorthi, Bilge Soran, and Vikas Chandra. Efficient track anything. *arXiv 2411.18933*, 2024.
 - [52] Jie Xu, Sangwoon Kim, Tao Chen, Alberto Rodriguez Garcia, Pulkit Agrawal, Wojciech Matusik, and Shinjiro Sueda. Efficient tactile simulation with differentiability for robotic manipulation. In *CoRL*, volume 205, pages 1488–1498, 2022.
 - [53] Sherry Yang, Yilun Du, Seyed Kamyar Seyed Ghasemipour, Jonathan Tompson, Leslie Pack Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. In *ICLR*, 2024.
 - [54] Andy Zeng, Shuran Song, Johnny Lee, Alberto Rodriguez, and Thomas A. Funkhouser. TossingBot: learning to throw arbitrary objects with residual physics. *IEEE T-RO*, 36(4):1307–1319, 2020.
 - [55] Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. VLABench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv 2412.18194*, 2024.
 - [56] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE RA-L*, 10(6):5209–5216, 2025.
 - [57] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.
 - [58] Kaizhi Zheng, Xiaotong Chen, Odest Chadwicke Jenkins, and Xin Eric Wang. VLMbench: A compositional benchmark for vision-and-language manipulation. In *NeurIPS*, 2022.
 - [59] Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, Xiaowei Zhang, Yuanfei Wang, Shaoyang Guo, Tianrui Guan, Ka Nam Lui, Zhiquan Qi, Yitao Liang, Yuanpei Chen, and Yaodong Yang. A survey on Vision-Language-Action models: An action tokenization perspective. *arXiv 2507.01925*, 2025.
 - [60] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted

Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, and Ayzaan Wahid et al. RT-2: Vision-Language-Action models transfer web knowledge to robotic control. In *CoRL*, 2023.

APPENDIX

A. Model Architecture Details

VLM Backbone. The RGB images in the temporal observation window $\mathbf{O}_t$ are concatenated and encoded by FastViT [44] using a hierarchical multi-stage design. Each input image is resized to 384×384 , and the encoder progressively increases channel width across stages (96, 192, 384, 768, 1536) with corresponding block depths (2, 12, 24, 4, 2). FastViT applies aggressive spatial compression via a large initial patch size of 64 and strided downsampling, using RepMixer-style token mixing in early stages and attention in later ones. The encoder outputs 36 visual tokens of fixed dimension 960, aligned with the language embedding space, achieving substantial token reduction while preserving manipulation-relevant spatial structure. In addition to visual inputs, the robot proprioceptive state $\mathbf{P}_t$ is incorporated as an explicit conditioning signal. The 32-dimensional state vector, containing Cartesian position and orientation with zero padding for unused entries, is linearly projected into the language embedding space and represented as a single 960-dimensional state token. Language instructions $\mathbf{L}_t$ are tokenized into a variable number of language tokens depending on prompt length. All visual, language, and state tokens are concatenated and processed jointly by the language backbone. Multimodal reasoning is performed by SmoLM2-360M [29], where only the first 16 transformer layers are used to reduce inference latency, following the practice adopted in SmoVLA [38]. The backbone outputs key-value representations for all processed tokens, which are cached and reused across inference cycles.

Action Expert. Action generation is handled by a dedicated diffusion-based action expert, instantiated as a lightweight transformer copied from the language backbone and truncated to the first 16 layers. The expert predicts an action chunk with horizon $n = 20$, which is sufficient under Continuous Inference while keeping inference latency low. Each action is a 32-dimensional vector representing end-effector pose and gripper state (with zero padding), and the noisy action input has shape $(n, 32)$ during training and pure noise during inference. The action expert uses a reduced hidden dimension of 720 ($0.75 \times$ the language embedding size) to lower computation. Noisy action tokens are projected into this space and combined with diffusion timestep embeddings, and denoising updates are generated by querying the cached key-value representations, without re-encoding perceptual inputs.

B. The Training Scheme

Pre-training Stage. The vision-language backbone combines a convolutional visual encoder (FastViT) and a compact language model (SmoLM2-360M), both initialized from their respective pretrained weights. To align visual and linguistic representations, we first perform large-scale vision-language pre-training using 150M English image-text pairs sampled from COYO-700M [8].

Mid-training Stage. After vision-language pre-training, the full VLA model is trained on the synthetic Dynamic Object Manipulation (DOM) dataset (Sec. IV). Each episode

TABLE III: **Ablation on Temporal Visual Context.** The temporal observation window is varied by enabling different visual frames at time steps $\{t-3, t-2, t-1, t\}$, while keeping the model architecture, inference frequency, and execution pipeline fixed. Note that SR, PL, T.Time, and I.Time represent the success rate (in %), path length (in meters), task completion time (in seconds), and inference time (in seconds, measured on an NVIDIA RTX A6000 GPU), respectively.

$t-3$	$t-2$	$t-1$	t	SR $\uparrow$	PL $\downarrow$	T.Time $\downarrow$	I.Time $\downarrow$
$\times$	$\times$	$\times$	$\checkmark$	38.22	2.27	9.52	0.225
$\times$	$\times$	$\checkmark$	$\checkmark$	43.39	2.34	8.77	0.226
$\times$	$\checkmark$	$\times$	$\checkmark$	47.06	2.50	8.53	0.226
$\checkmark$	$\times$	$\times$	$\checkmark$	46.89	2.49	8.51	0.226
$\times$	$\checkmark$	$\checkmark$	$\checkmark$	47.11	2.49	8.46	0.228
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	47.06	2.47	8.53	0.229

provides temporally evolving multi-view visual observations, from which the model uses a wrist-mounted camera on the end-effector and a fixed third-person camera facing the manipulator. To capture short-term dynamics, the temporal observation window is instantiated as $\mathbf{O}_t = \{\mathbf{o}_{t-2}, \mathbf{o}_t\}$. Using two views per timestep, this results in four images per input step, which are concatenated channel-wise and processed jointly by the vision encoder. In this stage, DynamicVLA is optimized using minibatches formed by randomly sampling episode timesteps from shuffled manipulation demonstrations. For each minibatch, the model is trained on tuples $(\mathbf{O}_t, \mathbf{L}_t, \mathbf{P}_t)$, while the action expert is trained to denoise a noisy action chunk $\mathbf{A}_t^\tau$ under the objective defined in Eq. 1.

Post-training stage. In the post-training stage, the model is fine-tuned on robot-specific real-world demonstrations using the same objective as in mid-training, enabling adaptation to new embodiments and sensing configurations.

C. Implementation Details

Training. DynamicVLA is trained on 32 NVIDIA A100 GPUs with a batch size of 40 per GPU. We use the AdamW optimizer with a learning rate of 1×10^{-4} , β coefficients (0.9, 0.95), $\epsilon = 1 \times 10^{-8}$, and weight decay of 1×10^{-10} . A cosine learning rate schedule with 1000 warm-up steps is employed. The models are trained for approximately two weeks, with three stages as 2 days for pre-training, 10 days for mid-training, and 2 days for post-training.

Inference. DynamicVLA requires 1.8GB of GPU memory and runs at approximately 88Hz on an NVIDIA RTX A6000 GPU.

D. More Discussion

Temporal Visual Context. We conduct an ablation study to analyze the impact of temporal visual context by varying the composition of the observation window $\mathbf{O}_t$ within the same DynamicVLA architecture. As described in Sec. III, our default setting feeds the model $\mathcal{M}$ with a sparse temporal window $\mathbf{O}_t = \{\mathbf{o}_{t-2}, \mathbf{o}_t\}$, which is designed to facilitate implicit object velocity perception. As summarized in Table III, different temporal configurations lead to negligible differences in inference latency and parameter count. We observe that using a single-frame input $\{\mathbf{o}_t\}$ results in a clear drop in task

TABLE IV: **Ablation on LLM Depth.** Different LLM depths are evaluated by retaining the first l transformer layers. Note that SR, PL, T.Time, I.Time, and #Param denote success rate (%), path length (meters), task completion time (seconds), inference time (seconds, measured on an NVIDIA RTX A6000 GPU), and parameter count (in millions), respectively.

#Layers	SR $\uparrow$	PL $\downarrow$	T.Time $\downarrow$	I.Time $\downarrow$	#Param $\downarrow$
8	44.17	2.33	8.92	0.127	303
16	47.06	2.50	8.53	0.226	430
24	48.44	2.63	8.43	0.317	558
32	42.11	2.69	8.39	0.373	685

TABLE V: **Cross-Model Analysis of CI and LAAS.** CI and LAAS are integrated into existing VLA models without backbone modification or retraining. Note that SR, PL, and Time represent the success rate (in %), path length (in meters), and task completion time (in seconds), respectively. $\dagger$ indicates inference-time integration of CI and LAAS.

Method	SR (%) $\uparrow$	PL (m) $\downarrow$	Time (s) $\downarrow$
$\pi_{0.5}^\dagger$ [15]	15.89	1.57	9.95
SmolVLA † [38]	25.56	1.65	9.77
DynamicVLA	47.06	2.50	8.53

success rate, as a single observation lacks the temporal cues necessary for estimating object motion and dynamics. However, expanding the temporal window beyond two frames does not yield further noticeable gains, indicating diminishing returns from additional visual redundancy. Moreover, compared to $\{\mathbf{o}_{t-2}, \mathbf{o}_t\}$, the setting $\{\mathbf{o}_{t-1}, \mathbf{o}_t\}$ achieves lower success rates, suggesting that a larger temporal interval provides more informative motion cues for velocity estimation. Overall, these results demonstrate that sparse but sufficiently spaced temporal context is critical for effective dynamic manipulation, even without increasing inference frequency.

Depth of LLM Backbone. Following the backbone truncation strategy [38], we reduce inference latency by retaining only the first l transformer layers of the LLM during inference. To examine whether this design choice remains effective in the DynamicVLA setting, we evaluate multiple backbone depths ($l = 8, 16, 24$) and compare them against the full model ($l = 32$). As shown in Table IV, increasing the backbone depth leads to a modest increase in inference latency. However, this additional latency can be largely amortized by Contiguous Inference and Latent-aware Action Streaming, and does not translate into a noticeable improvement in task success rate. In contrast, aggressively truncating the backbone significantly improves inference speed, but at the cost of reduced model capacity, resulting in a substantial degradation in success rate. Overall, this ablation confirms that a 16-layer backbone strikes the optimal balance between efficiency and robustness.

Cross-Model Analysis of CI and LAAS To evaluate the generality of the proposed execution mechanisms, Continuous Inference (CI) and Latent-aware Action Streaming (LAAS) are integrated into existing VLA models, including SmolVLA and $\pi_{0.5}$, without altering their backbone architectures. As shown

in Table V, consistent performance improvements are observed on SmolVLA, indicating that CI and LAAS effectively enhance closed-loop responsiveness under moderate inference latency. In contrast, $\pi_{0.5}$ exhibits only marginal gains, as its substantially larger backbone incurs high inference latency, which limits the effectiveness of overlapping inference and temporally aligned execution. Overall, these results suggest that CI and LAAS are broadly applicable execution mechanisms, while their practical benefits are constrained by the underlying inference latency of the model.

E. Detailed Evaluation Setup

In this section, we provide comprehensive details of the real-world evaluation setups used in our experiments, including task specifications and object configurations. Each task is executed under standardized conditions to ensure repeatability and fair comparison across different policies. Specifically, objects are launched by a secondary robot arm following a fixed trajectory, and evaluation is conducted across three predefined paired motion–position configurations, each combining an initial motion profile with a corresponding target container position.

Real-world Interaction Evaluation (Sec. V-B)

- **Place the coffee can into the wooden box.** The robot must track and grasp a rolling Nescafé coffee can and place it into a wooden box. This task evaluates closed-loop reactivity to continuously moving targets.
- **Place the conical bottle onto the frisbee.** The robot must grasp a conical roasted sesame bottle whose rolling motion follows a curved trajectory and place it onto a blue frisbee. This task evaluates closed-loop reactivity under non-linear object motion.
- **Place the pickleball into the paper box.** The robot must grasp a moving pickleball and place it into a paper box, where the ball is designed to collide with the box and undergo trajectory deflection. This task evaluates adaptive manipulation under contact-induced motion changes.
- **Place the ping pong ball inside the blue tape.** The robot must grasp a moving ping pong ball and place it within a blue-taped region, where impacts with the tape are designed to deflect the ball’s trajectory. This task evaluates adaptive placement under perturbed object motion.
- **Gather all ping pong balls into the paper box.** The robot must continuously collect ping pong balls that repeatedly appear on the tabletop and place them into a paper box. This task evaluates long-horizon task sequencing under sustained dynamic inputs.
- **Gather all tennis balls into the red tape.** The robot must continuously collect tennis balls that repeatedly appear on the tabletop and return them to a red-taped region. This task evaluates long-horizon planning and execution in dynamic environments.

Real-world Perception Evaluation (Sec. V-C)

- **Place the tennis ball into the paper bowl.** The robot must identify and grasp the moving tennis ball among

multiple simultaneously thrown objects (a tennis ball and a pickleball), and place it into a paper bowl. This task evaluates object-level visual understanding for identifying and manipulating the correct target under dynamic motion.

- **Place the tennis ball onto the blue-taped area.** The robot is required to catch a rolling tennis ball and place it precisely within the region marked by blue tape, among multiple visually similar tape markings (red, blue, and transparent). This task evaluates visually grounded target understanding and precise placement under continuous object motion.
- **Place the cola can on the left wooden box.** The robot must grasp a moving cola can and place it on a wooden box located to its left, evaluating its ability to handle spatial placement under varying object motions. This task evaluates spatial understanding for target localization and placement relative to the robot’s viewpoint.
- **Place the tennis ball on the right tape.** The robot must grasp a moving tennis ball and place it on a tape located to its right, evaluating spatial awareness and placement precision. This task evaluates spatial understanding for interpreting directionally specified targets and executing accurate placement.
- **Place the slower ball into the paper bowl.** The robot must grasp the ping pong ball specified by its lower moving speed and place it into the paper bowl. This task evaluates motion-based target understanding, where the target is specified by its movement direction.
- **Place the faster-rolling can inside the frisbee.** The robot must grasp the cola can specified by its higher rolling speed and place it inside the blue frisbee. This task evaluates motion-based target understanding, where the target is specified by its relative motion speed.

Real-world Generalization Evaluation (Sec. V-D)

- **Place the plastic bottle into the wooden box.** The robot must grasp a rolling plastic bottle with an unseen appearance and a regular curved trajectory, and place it into a wooden box. This task evaluates visual generalization to unseen object appearances under dynamic motion.
- **Place the golf ball in the red tape.** The robot must grasp a rolling golf ball with an unseen appearance and place it within a red-taped region. This task evaluates visual generalization to unseen object instances during dynamic manipulation.
- **Place the potato into the wooden box.** The robot must grasp a moving potato whose motion follows irregular patterns and place it into a wooden box. This task evaluates motion generalization to irregular object dynamics.
- **Place the green apple in the red tape.** The robot must grasp a moving green apple whose motion exhibits irregular and unpredictable patterns, and place it onto a red-taped region. This task evaluates motion generalization to irregular object trajectories.